CONCURRENT PROGRAMMING

CT074-3-2 CCP

INDIVIDUAL ASSIGNMENT

Name	**Low Yi Bo**
TP Number	**TP076096**
Lecturer	**TS. DR. MAYTHEM KAMAL ABBAS AL-ADILEE**
Intake	**APD2F2506CS(DA)**
Date Assigned	
Submission Date	

Table of Contents

1.0 Introduction and Background 3

1.1 System Overview 3

1.2 System Assumptions 3

2.0 Basic Requirements Met 4

2.1 Runway Mutual Exclusion Implementation 4

2.2 Ground Capacity Limitation Strategy 5

2.3 Aircraft Lifecycle & Passenger Concurrency 6

2.4 Simulated Operational Delays 9

2.5 ATC Ground Deadlock Prevention 10

2.6 Thread-Safe Statistical Tracking 11

3.0 Additional Requirements Met 12

3.1 Shared Refuelling Truck Resource 12

3.2 Congestion & Emergency Prioritization 13

4.0 Requirements Which Were Not Met 14

5.0 Conclusion 15

6.0 Appendix 16

1.0 Introduction and Background

1.1 System Overview

The Asia Pacific Airport simulation is a model for the simultaneous operations of a restricted-capacity airport. The facility consists of one shared runway, three docking gates and one refuelling truck. The system is designed to safely orchestrate the lifecycles of six airplanes which will be carrying as many as 50 passengers, as they request for landing, coast, dock, disembark, clean, refuel, board and eventually take off.

A crucial safety limit in the system is ground capacity limitation where it strictly limits a maximum number of 3 airplanes allowed on the airport grounds at any one time to avoid physical collisions. Furthermore, the system must be able to deal with complex and congested situations such as an emergency landing with a lack of fuel, which means they need priority routing over other waiting planes. The disciplinary point in this simulation is to show robust thread safety, deadlocks and thread starvation scenarios with concurrent manual Java synchronization constructs operations.

1.2 System Assumptions

To simulate the airport environment effectively, the following operational and technical assumptions were applied during development:

Assumption	Description
Time simulation scaling	Real-world action delays are scaled down by using `Thread.sleep()` to ensure the entire simulation completes within the 60 second limit.
Passenger readiness	Passengers are always ready at the terminal, meaning there is no secondary delays during boarding or disembarking.
Emergency trigger	Planes do not dynamically consume fuel in the air. The emergency plane is explicitly flagged `isEmergency = true` upon creation to trigger priority handling.

ATC queueing fairness	ATC enforces strict FIFO queuing to prevent thread starvation. The emergency plane bypasses this rule to jump to the front of the queue.
Sequential refuelling	The single refuelling truck services all the plane sequentially. Concurrent requests place waiting planes into a blocked state until the truck is free.

2.0 Basic Requirements Met

2.1 Runway Mutual Exclusion Implementation

In order to meet Basic Requirement 1, a single instance of the Runway class is created in the main() method and shared by all of the airplane threads. Because the runway is a shared resource, any change in its occupancy status is a critical section which needs to be protected from race conditions.

```
public static void main(String[] args) throws InterruptedException {

    // initialize shared resources
    runway = new Runway();
    airport = new Airport();
    gates = new Gate[gatesCount];
    cleaners = new Cleaning[gatesCount];
    cleanerThreads = new Thread[gatesCount];
```

Figure 1 Initialization of the Runway Object

To enforce mutual exclusion, the ReentrantLock is used in the Runway class. The occupy() method locks the runway to the currentPlane, while release() unlocks it. Essentially, the lock is initialized with a fairness policy set to true new ReentrantLock(true). This justification ensures strict First-In-First-Out (FIFO) queue, so that thread starvation is prevented as planes will get access to it in the same order as they requested to have accessed it. Furthermore, as a safety part, the lock.unlock() execution is placed within a finally block in the release() method. This ensures that the runway is never left behind even if an unexpected exception occurs or in effect prevents system deadlocks.

```java
public class Runway {

    //lock that ensure plane user the runway in order
    private final ReentrantLock lock = new ReentrantLock(true);
    private Plane currentPlane;

    //secure the runway for a specific plane
    public void occupy(Plane plane){
        lock.lock();
        try {
            currentPlane = plane;
        } finally {}
    }
    //free the runway for the next plane in queue
    public void release(){
        try {
            currentPlane = null;
        } finally {
            lock.unlock(); //unlocked when the plane safely leaves
        }
    }
```

Figure 2 ReentrantLock Implementation in Runway Class

2.2 Ground Capacity Limitation Strategy

To avoid collisions, the airport capacity is strictly limited to 3 planes using the Airport class.

A Semaphore with 3 permits controls the plane entry. The capacity.acquire() is used to safely block incoming planes when the airport is full and capacity.release() is used to free up a place when a plane takes off. The uses of new Semaphore(3, true) is because it creates a First-In-First-Out Box (FIFO)queue. This ensures that planes come in request order ensuring against starvation of threads.

to keep track of the planes, an AtomicInteger is used to track currentOccupancy. This approach is more inherently thread safe over a standard int. This avoids any race condition of concurrent capacity updates without the need of synchronized blocks.

```java
public class Airport {

    //maximum number of planes allowed in airport ground
    private static final int maxCapacity = 3;

    //semaphore controls real capacity
    private final Semaphore capacity = new Semaphore(maxCapacity, true);

    //thread safe counter to track the current plane count in airport
    private final AtomicInteger currentOccupancy = new AtomicInteger(0);

    // ATC calls this when granting landing clearance
    public void planeEntered() throws InterruptedException {
        capacity.acquire();              // block if full
        currentOccupancy.incrementAndGet();
    }
    // Plane calls this when fully leaving airport
    public void planeLeft() {
        currentOccupancy.decrementAndGet(); //release the permit to wake up waiting threads
        capacity.release();
    }

    //getter
    public int getCurrentOccupancy() {
        return currentOccupancy.get();
    }
    public int getMaxCapacity() {
        return maxCapacity;
    }

    //condition checker
    public boolean hasSpace() {
        return capacity.availablePermits() > 0;
    }
    public boolean isFull() {
        return capacity.availablePermits() == 0;
    }
```

Figure 3 Airport Capacity Control Using Semaphore

2.3 Aircraft Lifecycle & Passenger Concurrency

To meet Basic Requirement 3, the lifecycle of every airplane runs sequentially in its run() method.

For the gate operations, the system is optimized by applying one thread to represent the entire group of passengers, as opposed to producing a thread for every passenger.

As a key safety measure, Thread.join() method is used to manage these passenger threads. join() is used to safely blocks the main plane thread until the passenger group thread finishes its task. This strict thread coordination safely prevents race conditions, guaranteeing the plane never undocks while passengers are still boarding or disembarking without needing complex manual wait/notify locks.

```java
public void run(){
    try{
        arrivalTime = System.currentTimeMillis();
        requestLanding();
        land();
        coastToGate();
        dock();
        performGateOperation();
        undock();
        coastToRunway();
        takeOff();

        Main.statistics.recordPlane(this);

    } catch(InterruptedException e){
        System.out.println(planeName + " was interrupted.");
        Thread.currentThread().interrupt();
    }
}
```

Figure 4 Plane run() method

```java
private void requestLanding() throws InterruptedException{
    //print the emergency tag
    if (fuelShortage) {
        System.out.println(planeName + ": Requesting Landing. [EMERGENCY]");
    } else {
        System.out.println(planeName + ": Requesting Landing.");
    }

    assignedGate = Main.atc.requestLandingPermission(this);
    landingTime = System.currentTimeMillis();
}
```

Figure 5 requestLanding()

```java
private void land() throws InterruptedException{
    System.out.println(planeName + ": Landing.");
    Main.runway.occupy(this);
    Thread.sleep(1000);
    System.out.println(planeName + ": Landed.");
}
```

Figure 6 land()

```java
private void coastToGate() throws InterruptedException{
    System.out.println(planeName + ": Coasting to Gate-" + assignedGate.getGateId());
    Main.runway.release();
    Thread.sleep(500);
}
```

Figure 7 coastToGate()

```java
private void dock() throws InterruptedException{
    assignedGate.dock(this);
    System.out.println(planeName + ": Docked at Gate-" + assignedGate.getGateId());
}
```

Figure 8 dock()

```java
private void performGateOperation()throws InterruptedException{
    // 1. unload arrival passengers
    disembark();

    // 2. concurrent services
    System.out.println(planeName + ": Requesting Cleaning at Gate-" + assignedGate.getGateId());
    assignedGate.requestCleaning(this); // Tell cleaners to start

    refuelPlane(); // Go get fuel at the same time

    assignedGate.waitForCleaning(); // Make sure cleaners are done before boarding

    // 3. load new departing passengers
    boardPassengers();
}
```

Figure 9 performGateOperation()

```java
private void undock() throws InterruptedException{
    System.out.println(planeName + ": Undock from Gate-" + assignedGate.getGateId());
    assignedGate.undock();
}
```

Figure 10 undock()

```java
private void coastToRunway() throws InterruptedException{
    System.out.println(planeName + ": Coasting to Runway.");
    Thread.sleep(500);
}
```

Figure 11 coastToRunway()

```java
private void takeOff() throws InterruptedException{
    System.out.println(planeName + ": Requesting Taking off.");
    Main.atc.requestTakeOffPermission(this);
    Main.runway.occupy(this);
    System.out.println(planeName + ": Taking-off.");
    Thread.sleep(1000);
    Main.runway.release();
    Main.airport.planeLeft();
}
```

Figure 12 takeOff()

```java
private void disembark() throws InterruptedException {
    Passenger disembarkingGroup = new Passenger(passengerCount, "Disembarking");

    Thread passengerThread = new Thread(disembarkingGroup, planeName);
    passengerThread.start();

    passengerThread.join();
}
```

Figure 13 disembark()

```java
private void refuelPlane() throws InterruptedException{
    System.out.println(planeName + ": Request refueling.");
    Main.refuelingTruck.refuel(this);
    System.out.println(planeName + ": Refueling completed.");
}
```

Figure 14 refuelPlane()

```java
private void boardPassengers() throws InterruptedException {
    this.passengerCount = random.nextInt(50) + 1;

    Passenger boardingGroup = new Passenger(passengerCount, "Boarding");

    Thread passengerThread = new Thread(boardingGroup, planeName);
    passengerThread.start();

    passengerThread.join();
}
```

Figure 15 boardPassengers()

2.4 Simulated Operational Delays

In order to meet the Basics of Requirement 4, some artificial time delays are imposed at each operating stage using Thread.sleep(). This ensures that actions such as landing and refuelling don't happen instantaneously.

Implementing this method is justified over a busy waiting loop because it is quite efficient in managing system resources. When the sleep() method is called, it takes current thread to waiting state for a duration of time. This is a critical aspect from a performance and safety point of view as it allows temporarily to yield the processor to other threads which are active. Consequently, this system does not waste unnecessary processing power, and it is able to avoid processor exhaustion, and correctly emulate real world operational delays.

```java
private void land() throws InterruptedException{
    System.out.println(Thread.currentThread().getName() + ":Landing.");
    Main.runway.occupy(this);
    Thread.sleep(1000);
    System.out.println(Thread.currentThread().getName() + ":Landed.");
}
```

Figure 16 Thread.sleep()

2.5 ATC Ground Deadlock Prevention

In order to meet the basic requirement number 5, the system uses a centralised controller to avoid aircraft waiting on the ground for a designated gate. When an airplane needs the landing clearance, it calls the requestLandingPermission method that places the current airplane in a landing queue and safely retains the thread using the wait method. This implementation is justified as an effective deadlock prevention strategy. Instead of allowing planes to land and block the airport grounds indefinitely, the controller continuously monitors all shared resources. It only provides clearance and wakes up a suspended plane thread by the notify method when both physical runway and a gate slot are explicitly available. By enforcing this strict air holding pattern the system guarantees safe allocation of resources and totally eliminates the risk of congestion on the ground and system wide deadlocks.

```java
public class ATC implements Runnable {

    //queueing list for landing and takeoff sequence
    private final LinkedList<Plane> landingQueue;
    private final LinkedList<Plane> takeoffQueue;
```

Figure 17 ATC Landing and Takeoff Queue

```java
public void run(){
    try{
        arrivalTime = System.currentTimeMillis();
        requestLanding();
        land();
        coastToGate();
        dock();
        performGateOperation();
        undock();
        coastToRunway();
        takeOff();

        Main.statistics.recordPlane(this);
```

Figure 18 Plane Operational Sequence

```java
    public Gate requestLandingPermission(Plane plane) throws InterruptedException {
        synchronized (atcLock) {
            landingQueue.add(plane);
            atcLock.notifyAll();
        }
        synchronized (plane) {
            while (plane.getAssignedGate() == null) {
                plane.wait();
            }
            return plane.getAssignedGate();
        }
    }
```

Figure 19 Landing Request and Waiting Mechanism

```java
if (!Main.airport.hasSpace() || gateSlots.availablePermits() == 0) {
    if (landingPlane.getPlaneId() != lastDeniedId) {
        System.out.println(Thread.currentThread().getName() + ": Landing Permission Denied for Plane=" + landingPlane.getPlaneId() + ". Airport/Gates Full.");
        lastDeniedId = landingPlane.getPlaneId();
    }
}
// runway status
else if (!Main.runway.isAvailable()) {
    if (landingPlane.getPlaneId() != lastDeniedId) {
        System.out.println(Thread.currentThread().getName() + ": Landing Permission Denied for Plane=" + landingPlane.getPlaneId() + ". Runway Occupied.");
        lastDeniedId = landingPlane.getPlaneId();
    }
```

Figure 20 Landing Permission Control Logic

2.6 Thread-Safe Statistical Tracking

To meet basic requirement 6, the system keeps track of waiting times, cumulative passengers, and planes served. The printStatistic method is a thread safe approach to reporting these results. Since more than one thread is used for updating data, the system has a synchronized block with a statsLock. This is a very important aspect of safety that can only have 1 thread modifying the shared data at a time. This helps in avoiding race conditions and helps make sure that the final output is accurate and free from data corruption.

```java
    public void printStatistic(){
        synchronized(statsLock){
            System.out.println("SIMULATION STATISTICS");

            if(waitingTimes.isEmpty()){
                System.out.println("No data collected");
                return;
            }

            long maxWaitTime = Collections.max(waitingTimes);
            long minWaitTime = Collections.min(waitingTimes);
            double avgWaitTime = waitingTimes.stream().mapToLong(Long::longValue).average().orElse(0);

            System.out.println("Waiting Time Statistic:");
            System.out.println("Max:" + maxWaitTime);
            System.out.println("Min:" + minWaitTime);
            System.out.println("Avg:" + String.format("%.1f", avgWaitTime));

            System.out.println("Service Statistics:");
            System.out.println("Total planes served: " + totalPlanesServed);
            System.out.println("Total passengers boarded: " + totalPassengersBoarded);

            if (totalPlanesServed > 0) {
                double avgPassengers = (double) totalPassengersBoarded / totalPlanesServed;
                System.out.println("Average passengers per plane: " + String.format("%.1f", avgPassengers));
            }
        }
    }
```

Figure 21 Simulation Statistics Calculation

3.0 Additional Requirements Met

3.1 Shared Refuelling Truck Resource

In order to meet the additional requirement 1, an independent refuelling truck is used as a separate thread in the system to simulate real world operations. The main method creates 1 instance of the RefuelingTruck class and has it running as a thread. When an airplane thread needs service, it calls the refuel method in which access is taken with the help of a ReentrantLock. The reason for using the lockInterruptibly method is that it is more flexible than the synchronized block because in this way the threads can get interrupted while waiting for the resource. This implementation is a critical part of the safety in that only 1 plane can have the truck as its currentPlane at any given time. By enforcing this strict mutual exclusion, the system avoids the possibility that more than one aircraft tries to use the single refuelling resource at the same time, which would avoid race conditions and ensures thread safe sequential servicing.

```java
    public static void main(String[] args) throws InterruptedException {

        // initialize shared resources
        runway = new Runway();
        airport = new Airport();
        gates = new Gate[gatesCount];
        cleaners = new Cleaning[gatesCount];
        cleanerThreads = new Thread[gatesCount];

        for(int i = 0; i < gatesCount; i++){
            gates[i] = new Gate();
            // Initialize a cleaning team for each gate
            cleaners[i] = new Cleaning(gates[i]);
            cleanerThreads[i] = new Thread(cleaners[i], "Cleaner-Gate" + (i+1));
        }

        statistics = new Statistic();
        refuelingTruck = new RefuelingTruck();
        atc = new ATC();

        // start threads
        Thread atcThread = new Thread(atc, "ATC");
        atcThread.start();

        Thread truckThread = new Thread(refuelingTruck, "RefuelingTruck");
        truckThread.start();
```

Figure 22 Refuelling Truck Initialization

```java
public void refuel(Plane plane) throws InterruptedException {
    truckLock.lockInterruptibly();
    try {
        // 1. Hand off the task
        this.currentPlane = plane;
        this.isRefueling = true;

        // 2. Wait for the Truck Thread to finish the work
        while (this.isRefueling) {
            Thread.sleep(50);
        }
    } finally {
        truckLock.unlock();
    }
}
```

Figure 23 Refuelling Control Mechanism

3.2 Congestion & Emergency Prioritization

To satisfy additional requirement 2, the system simulates a busy scenario by starting 6 plane threads against a limit of only 3 gates. An emergency landing is discovered by checking the below specific aircraft by using the hasFuelShortage method. In the main control loop of the ATC, it checks the landingQueue to identify if any aircrafts have shortage of fuel. This approach is justified as an aspect of safety of prime importance when managing priorities. If an emergency is found, the controller selects that plane for immediate landing overriding the other standard practice of landing in first in first out order. This priority logic ensures that any critical events are fulfilled with the highest level of urgency. It effectively avoids thread starvation to emergency aircraft and guarantees the positive resolution of high risk scenarios at times of high airport congestion.

```java
public class Main {
    /**
     * @param args the command line arguments
     */

    public static Runway runway;
    public static Airport airport;
    public static ATC atc;
    public static RefuelingTruck refuelingTruck;
    public static Gate[] gates;
    public static Statistic statistics;

    // Arrays for the cleaning teams
    public static Cleaning[] cleaners;
    public static Thread[] cleanerThreads;

    private static final int gatesCount = 3;
    private static final int numPlanes = 6;
    private static final int emergencyPlaneId = 5;
```

Figure 24 Congested Scenario Configuration

```java
// planes 6
Thread[] planeThreads = new Thread[numPlanes];
Random rand = new Random();
for (int i = 1; i <= numPlanes; i++) {
    boolean isEmergency = (i == emergencyPlaneId);
    Plane plane = new Plane(isEmergency);
    Thread planeThread = new Thread(plane, "Plane-" + i);
    planeThreads[i - 1] = planeThread;
    planeThread.start();
    Thread.sleep(rand.nextInt(2001));
}
```

Figure 25 Plane Thread Initialization with Emergency Flag

```java
while (landingQueue.isEmpty() && takeoffQueue.isEmpty() && running) {
    atcLock.wait();
}
if (!running) break;

if (!landingQueue.isEmpty()) {
    for (Plane p : landingQueue) {
        if (p.hasFuelShortage()) {
            landingPlane = p;
            break;
        }
    }
    if (landingPlane == null) {
        landingPlane = landingQueue.getFirst();
    }
}
```

Figure 26 Emergency Landing Priority Selection Logic

4.0 Requirements Which Were Not Met

All basic and additional requirements as determined in the project brief were successfully realized without an omission. The system successfully handles the 1 common runway, 3 airplanes ground capacity limit, and complete sequential lifecycle of each airplane. The single refuelling truck and the emergency landing priority logic also work within the required parameters so that. There are no known requirements that were not met, for the simulation being able to successfully demonstrate thread safety and deadlock prevention in all mandatory and optional scenarios.

5.0 Conclusion

The simulation of the Asia Pacific Airport is able to show the practical application of the fundamental principles of concurrent programming. By the implementation of manual Java synchronization utilities with ReentrantLock for runways mutual exclusion, Semaphore for strict control of ground capacity, synchronized blocks for shared resource management, the system orchestrates safely the complex life-cycles of the numerous strands of passengers and aircraft.

Crucially, the architecture uses active prevention in the avoidance of race conditions, thread starvation and deadlocks. The ATC routing logic allows planes not to wait indefinitely on the tarmac, while the prioritized queueing system powerfully handles congested emergency landing scenarios, and exclusive access to the refuelling truck. Ultimately, the simulation meets with all the given basic and additional requirements, leading to a highly robust and thread-safe system that processes the dynamic concurrent events efficiently within the rigid 60 seconds execution window.

6.0 Appendix